

Sudoku Solver using Backtracking

Language Used: Python

Prepared by: Tanya Donepudi

Registration No: AP24110011492

Course/Branch: Computer Science and Engineering

Section: T

Date: May 03, 2026

1. Abstract

This report details the design and implementation of a Sudoku solver utilizing the **backtracking algorithm**. The problem of solving Sudoku puzzles, a popular logic-based number-placement puzzle, is framed as a computational challenge. The backtracking approach systematically explores potential solutions, validating constraints at each step to ensure correctness. Key outcomes include an efficient Python implementation capable of solving 9x9 Sudoku grids and a comprehensive analysis of its performance characteristics.

2. Introduction

Sudoku is a combinatorial number-placement puzzle. The objective is to fill a 9x9 grid with digits so that each column, each row, and each of the nine 3x3 subgrids that compose the grid contain all of the digits from 1 to 9. The puzzle setter provides a partially completed grid, which for a well-posed puzzle has a single solution.

Solving Sudoku is a classic example of a computational problem that benefits from algorithmic thinking. It involves searching for a valid configuration within a constrained search space. The importance of algorithmic thinking extends beyond puzzles, forming the foundation for solving complex problems in computer science, artificial intelligence, and operations research.

3. Problem Statement

To design and implement an algorithm that solves a 9x9 Sudoku puzzle efficiently.

4. Objectives

- Implement the **backtracking algorithm** for solving Sudoku puzzles.

- Develop robust functions for **validating constraints** (row, column, and subgrid rules).
- Generate the **correct and unique solution** for any given valid Sudoku puzzle.

5. Methodology

The **backtracking algorithm** is a general algorithmic technique for solving problems recursively by trying to build a solution incrementally, one piece at a time, removing those solutions that fail to satisfy the constraints of the problem at any point of time. For Sudoku, this involves:

- 1 **Finding an Empty Cell:** The algorithm starts by locating an empty cell (represented by 0) in the Sudoku grid.
- 2 **Trying Possible Numbers:** For each empty cell, it attempts to place a digit from 1 to 9.
- 3 **Constraint Checking:** Before placing a digit, it checks if the digit is valid according to Sudoku rules (i.e., not present in the current row, column, or 3x3 subgrid).
 - **Row Validation:** Ensures the digit is not already present in the current row.
 - **Column Validation:** Ensures the digit is not already present in the current column.
 - **Subgrid Validation:** Ensures the digit is not already present in the 3x3 subgrid corresponding to the current cell.
- 4 **Recursion:** If a digit is valid, it is placed in the cell, and the solve_sudoku function is called recursively for the next empty cell. This represents moving deeper into the solution space.
- 5 **Backtracking:** If the recursive call returns False (meaning no valid digit could be placed in a subsequent cell), the current digit is removed (the cell is reset to 0), and the algorithm tries the next possible digit for the current cell. This is the 'backtracking' step, where the algorithm reverts to a previous state to explore alternative paths.
- 6 **Solution Found:** If all cells are filled without violating any rules, a solution has been found, and the algorithm returns True.

6. Algorithm

Pseudocode for solve_sudoku(board)

```
function solve_sudoku(board):

    find = find_empty(board)
    if not find:
        return True // No empty cell, puzzle solved
    else:
        row, col = find

        for num from 1 to 9:
```

```

        if is_valid(board, num, (row, col)):
            board[row][col] = num

            if solve_sudoku(board):
                return True // Solution found recursively

            board[row][col] = 0 // Backtrack: reset cell

    return False // No solution for this path

```

Pseudocode for is_valid(board, num, pos)

```

function is_valid(board, num, pos):

    // Check row
    for each column i in board[pos.row]:
        if board[pos.row][i] == num and i != pos.col:
            return False

    // Check column
    for each row i in board:
        if board[i][pos.col] == num and i != pos.row:
            return False

    // Check 3x3 box
    box_x = pos.col // 3
    box_y = pos.row // 3

    for row_in_box from box_y * 3 to box_y * 3 + 2:
        for col_in_box from box_x * 3 to box_x * 3 + 2:
            if board[row_in_box][col_in_box] == num and (row_in_box,
col_in_box) != pos:
                return False

    return True

```

7. Implementation

The Sudoku solver is implemented in Python. The core logic resides in three functions: solve_sudoku, is_valid, and find_empty. The print_board function is used for displaying the Sudoku grid.

```

def solve_sudoku(board):

```

```

find = find_empty(board)
if not find:
    return True
else:
    row, col = find

    for i in range(1, 10):
        if is_valid(board, i, (row, col)):
            board[row][col] = i

            if solve_sudoku(board):
                return True

            board[row][col] = 0

    return False

def is_valid(board, num, pos):
    # Check row
    for i in range(len(board[0])):
        if board[pos[0]][i] == num and pos[1] != i:
            return False

    # Check column
    for i in range(len(board)):
        if board[i][pos[1]] == num and pos[0] != i:
            return False

    # Check 3x3 box
    box_x = pos[1] // 3
    box_y = pos[0] // 3

    for i in range(box_y * 3, box_y * 3 + 3):
        for j in range(box_x * 3, box_x * 3 + 3):
            if board[i][j] == num and (i, j) != pos:
                return False

    return True

def find_empty(board):
    for r in range(len(board)):
        for c in range(len(board[0])):
            if board[r][c] == 0:
                return (r, c) # row, col

    return None

def print_board(board):
    for r in range(len(board)):
        if r % 3 == 0 and r != 0:
            print("- - - - -")

```

```

for c in range(len(board[0])):
    if c % 3 == 0 and c != 0:
        print(" | ", end="")

    if c == 8:
        print(board[r][c])
    else:
        print(str(board[r][c]) + " ", end="")

if __name__ == "__main__":
    # Sample Sudoku puzzle (0 represents empty cells)
    board = [
        [5, 3, 0, 0, 7, 0, 0, 0, 0],
        [6, 0, 0, 1, 9, 5, 0, 0, 0],
        [0, 9, 8, 0, 0, 0, 0, 6, 0],
        [8, 0, 0, 0, 6, 0, 0, 0, 3],
        [4, 0, 0, 8, 0, 3, 0, 0, 1],
        [7, 0, 0, 0, 2, 0, 0, 0, 6],
        [0, 6, 0, 0, 0, 0, 2, 8, 0],
        [0, 0, 0, 4, 1, 9, 0, 0, 5],
        [0, 0, 0, 0, 8, 0, 0, 7, 9]
    ]

    print("Unsolved Sudoku:")
    print_board(board)
    print("\n")

    if solve_sudoku(board):
        print("Solved Sudoku:")
        print_board(board)
    else:
        print("No solution exists.")

```

- solve_sudoku(board): This is the main recursive function that implements the backtracking logic. It finds an empty cell, tries numbers from 1 to 9, and recursively calls itself.
- is_valid(board, num, pos): Checks if placing num at pos (row, column) is valid according to Sudoku rules.
- find_empty(board): Searches the board for an empty cell (value 0) and returns its coordinates.
- print_board(board): A utility function to display the Sudoku board in a readable format.

8. Sample Input and Output

Unsolved Sudoku

5	3		.	.	7		.	.	.
6	.		1	9	5		.	.	.
.	9	8		.	.		.	6	.
-	-	-	-	-	-	-	-	-	-
8	.		.	6	.		.	.	3
4	.		8	.	3		.	.	1
7	.		.	2	.		.	.	6
-	-	-	-	-	-	-	-	-	-
.	6		.	.	.		2	8	.
.	.		4	1	9		.	.	5
.	.		.	8	.		.	7	9

Solved Sudoku

5	3	4		6	7	8		9	1	2
6	7	2		1	9	5		3	4	8
1	9	8		3	4	2		5	6	7
-	-	-	-	-	-	-	-	-	-	-
8	5	9		7	6	1		4	2	3
4	2	6		8	5	3		7	9	1
7	1	3		9	2	4		8	5	6
-	-	-	-	-	-	-	-	-	-	-
9	6	1		5	3	7		2	8	4
2	8	7		4	1	9		6	3	5
3	4	5		2	8	6		1	7	9

9. Results and Analysis

Time Complexity

The time complexity of a backtracking Sudoku solver is difficult to determine precisely due to its recursive nature and dependency on the puzzle's initial state. In the worst-case scenario, for an empty 9x9 grid, the algorithm might explore 9^{81} possibilities, which is astronomically large. However, with effective pruning (constraint checking), the actual performance is significantly better. For typical Sudoku puzzles, the algorithm performs well because many branches of the search tree are eliminated early. The complexity can be approximated as $O(9^{(N*N)})$ where N is the size of the grid (N=9 for standard Sudoku), but practically, it's much faster due to pruning.

Space Complexity

The space complexity is primarily determined by the recursion stack. In the worst case, the `solve_sudoku` function might be called recursively for each of the 81 cells. Therefore, the space complexity is $O(N*N)$ or $O(81)$ for a 9x9 grid, representing the depth of the recursion stack.

Observations

- The backtracking algorithm effectively finds a solution if one exists.
- The efficiency is highly dependent on the initial number of pre-filled cells; more pre-filled cells lead to faster solutions.
- The `is_valid` function is critical for pruning the search space and improving performance.

10. Advantages

- **Simple and Effective:** The logic is straightforward to understand and implement.
- **Guarantees Correct Solution:** If a solution exists, the backtracking algorithm is guaranteed to find it because it explores all possible valid paths.
- **Versatile:** The core backtracking principle can be applied to a wide range of constraint satisfaction problems.

11. Limitations

- **Can be Slow for Complex Puzzles:** For puzzles with very few initial clues or particularly challenging configurations, the algorithm can take a considerable amount of time to find a solution.
- **Not Optimized for Speed:** While effective, it's not the fastest algorithm for Sudoku. More advanced techniques like constraint propagation (e.g., Dancing Links algorithm) can solve puzzles much faster.
- **No Learning:** The algorithm does not 'learn' from previous attempts or puzzle structures; it's a brute-force search with intelligent pruning.

12. Applications

- **Puzzle Solving:** Beyond Sudoku, backtracking is used for N-Queens problem, Knight's Tour, crossword puzzles, and other combinatorial puzzles.
- **Constraint Satisfaction Problems (CSPs):** A fundamental technique for solving problems where solutions must satisfy a set of constraints, common in AI and operations research.

- **AI Problem Solving Basics:** Serves as an excellent introductory example for understanding search algorithms, recursion, and problem-solving strategies in Artificial Intelligence.

13. Conclusion

This project successfully implemented a Sudoku solver using the backtracking algorithm in Python. The exercise demonstrated the power of recursive problem-solving and the importance of constraint validation in pruning search spaces. Backtracking is a foundational algorithm in computer science, offering a systematic approach to problems that involve exploring multiple possibilities under specific rules. Understanding its mechanics provides valuable insights into more complex AI and optimization techniques.

14. Future Enhancements

- **GUI Visualization:** Develop a graphical user interface to visualize the solving process step-by-step, enhancing user understanding and engagement.
- **Optimization Techniques:** Implement more advanced algorithms like Dancing Links (Algorithm X) or incorporate heuristics to significantly improve solving speed for difficult puzzles.
- **Real-time Solver:** Integrate the solver into a web application or mobile app for real-time Sudoku solving, potentially using image recognition for input.

16. References

- [1] [Russell, S. J., & Norvig, P. \(2010\). Artificial Intelligence: A Modern Approach \(3rd ed.\). Prentice Hall.](#)
- [2] [Knuth, D. E. \(2000\). Dancing Links. Millennial Perspectives in Computer Science, 1, 1-21.](#)
- [3] [Wikipedia. \(n.d. \). Sudoku. Retrieved from](#)